

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL3_Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA1522

Name	Dinnepati Sindhu Prasad
Register Number	192311271

COURSE OUTCOME COVERED

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct

I Dinnepati Sindhu Prasad-192311271 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Dinnepati Sindhu Prasad

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.
2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.

3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage.

Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.

5. A development team wants to adopt containerization for application deployment.

Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

1. Auto-Scaling and Load Balancing for Seasonal Traffic Spikes

How auto-scaling and load balancing handle demand:

Cloud auto-scaling automatically adjusts the number of active compute instances (VMs or containers) based on real-time demand metrics like CPU utilization, memory usage, or request count. During a sale event, when traffic surges, the auto-scaling group spins up additional instances within minutes; when traffic drops back to normal, it scales down to avoid wasting resources. Load balancers (like AWS ELB, Azure Load Balancer, or NGINX-based solutions) sit in front of these instances and distribute incoming requests evenly, preventing any single server from becoming a bottleneck.

Reducing downtime and improving user experience:

Without auto-scaling, a fixed-capacity server would either be over-provisioned (wasting money most of the year) or under-provisioned (crashing during peak load). Auto-scaling combined with load balancing means the system dynamically matches capacity to demand, so pages load quickly, checkout doesn't fail, and the site stays available even during flash sales. Load balancers also perform health checks, automatically routing traffic away from unhealthy instances, which prevents user-facing errors.

Role of monitoring tools:

Monitoring tools (CloudWatch, Azure Monitor, Google Cloud Operations Suite) are what make auto-scaling "smart." They continuously track metrics and trigger scaling policies when thresholds are crossed (e.g., "add 2 instances if CPU > 70% for 5 minutes"). They also provide dashboards and alerts so engineers can spot anomalies, track latency, and audit post-event performance.

Real-world example:

Amazon during Black Friday/Prime Day is the classic case — traffic can jump 5–10x normal levels within hours. Auto-scaling groups behind Elastic Load Balancers absorb this surge, while CloudWatch alarms trigger scale-out events automatically. Flipkart's Big Billion Days sale in India is a comparable domestic example, where cloud elasticity prevents the site crashes that plagued earlier, non-cloud-native versions of such events.

2. Secure Cloud Backup Storage

Durability and redundancy:

Cloud storage services (Amazon S3, Azure Blob Storage, Google Cloud Storage) achieve high durability (often quoted as 99.999999999% / "11 nines") by automatically replicating data across multiple physical devices, and often across multiple availability zones or geographic regions. This means even if a disk, server rack, or entire data center fails, copies of the data remain intact elsewhere. Techniques like erasure coding and multi-AZ replication are used instead of relying on a single copy, which traditional on-premise tape or disk backups often did.

Encryption and access control:

- **Encryption at rest** ensures stored backup data is unreadable without the correct key, even if physical storage media is compromised.
- **Encryption in transit** (TLS) protects data as it moves between the client and cloud storage.
- **Access control** via IAM (Identity and Access Management) policies restricts who can read, write, or delete backups — following the principle of least privilege. Multi-factor authentication and role-based access add further layers.
- **Object versioning and immutability locks** (like S3 Object Lock) can prevent ransomware or accidental deletion from destroying backups.

Cost advantages over traditional storage:

Traditional storage requires upfront capital expenditure (buying servers, tape drives, data center space) plus ongoing maintenance, cooling, and staffing costs — regardless of whether the capacity is fully used. Cloud storage uses a pay-as-you-go model, and tiered storage classes (e.g., S3 Standard vs. S3 Glacier) let organizations move infrequently accessed backups to much cheaper "cold" storage automatically, cutting costs significantly for data that's rarely retrieved.

Disaster recovery example:

A hospital storing patient records could use cloud backup with cross-region replication. If a fire or flood destroys the primary data center, the organization can restore operations from a geographically separate region within hours instead of days, minimizing both data loss and operational disruption — critical when patient care depends on record availability.

3. CDNs for Global Mobile App Deployment

How CDNs improve performance and reduce latency:

A CDN is a globally distributed network of servers that cache and serve content (images, videos, static assets, sometimes even dynamic content) from a location physically closer to the end user. Instead of every request traveling back to a single origin server (say, in the US), a user in India gets served from a nearby edge node, drastically cutting round-trip time.

Role of edge servers:

Edge servers store cached copies of content and can also perform edge computing — running lightweight logic (authentication checks, image resizing, A/B testing) closer to the user. This reduces the load on origin servers and improves responsiveness, especially for latency-sensitive interactions like button taps or live updates in a mobile app.

How cloud providers support global availability:

Major providers (AWS CloudFront, Google Cloud CDN, Azure CDN, Cloudflare) operate points of presence (PoPs) across dozens of countries. Combined with global load balancing (routing users to the nearest healthy region) and multi-region deployments, cloud providers let a company serve a consistent experience worldwide without manually managing infrastructure in every country.

Real-time streaming/gaming example:

Netflix uses its own CDN (Open Connect) embedded within ISP networks worldwide, so video streams from servers extremely close to the viewer, minimizing buffering. In gaming,

companies like Riot Games (League of Legends) deploy regional servers and use CDN-backed infrastructure to reduce ping for real-time multiplayer sessions — where even 50ms of extra latency can noticeably affect gameplay.

4. Cloud Compliance and Security for Financial Institutions

Supporting regulatory compliance and auditing:

Cloud providers offer compliance certifications (ISO 27001, SOC 2, PCI-DSS, GDPR-readiness) and provide tools that map directly to regulatory requirements — such as AWS Artifact or Azure Compliance Manager, which give auditable documentation. Services like AWS CloudTrail or Azure Activity Log record every API call and configuration change, creating an immutable audit trail regulators can review.

Role of logging, monitoring, and encryption:

- **Logging** captures who accessed what data and when, essential for forensic investigation and compliance audits (e.g., RBI or SEBI requirements in India, or SOX/PCI-DSS internationally).
- **Monitoring** tools detect anomalous behavior in real time — e.g., an account suddenly accessing large volumes of transaction data outside business hours can trigger alerts.
- **Encryption** (both at rest and in transit) ensures that even if data is intercepted or storage is breached, sensitive financial information (account numbers, transaction histories) remains unreadable without proper keys, which are often managed via hardware security modules (HSMs) or cloud KMS (Key Management Service).

Meeting data sovereignty requirements:

Financial regulators often require that customer data physically reside within national borders. Cloud providers address this by offering region-specific data centers (e.g., AWS's Mumbai region, Azure's India regions) and letting organizations pin storage and processing to a specific geography, ensuring data never leaves the required jurisdiction.

Example:

A bank processing UPI or card transactions might use a private, region-locked cloud environment in Mumbai, with all transaction logs encrypted using KMS-managed keys, CloudTrail logging every administrative action, and automated compliance reports generated for RBI audits — ensuring both security and regulatory adherence without manual overhead.

5. Containerization and Orchestration for Deployment

Containers vs. virtual machines:

VMs virtualize an entire hardware stack, each running its own full OS via a hypervisor — this makes them heavier (gigabytes in size) and slower to start (minutes). Containers, by contrast, virtualize at the OS level, sharing the host kernel while isolating the application and its dependencies. This makes containers lightweight (megabytes), fast to start (seconds), and far more resource-efficient, since dozens of containers can run on a host that might support only a handful of VMs.

How orchestration (Kubernetes) simplifies deployment and scaling:

Managing hundreds of containers manually is impractical. Kubernetes automates:

- **Scheduling** — deciding which nodes run which containers based on resource availability.
- **Self-healing** — automatically restarting failed containers or rescheduling them on healthy nodes.
- **Scaling** — horizontally scaling pods up or down based on load (similar in spirit to auto-scaling in Question 1).
- **Service discovery and load balancing** — routing traffic to healthy container instances automatically.
- **Rolling updates and rollbacks** — deploying new versions with zero downtime, and reverting quickly if something breaks.

Portability benefits:

Because containers package the application with all its dependencies, they run consistently across environments — a developer's laptop, an on-prem server, or any major cloud provider (AWS, Azure, GCP). This avoids "it works on my machine" issues and prevents vendor lock-in, since Kubernetes itself is cloud-agnostic and supported by all major providers (EKS, AKS, GKE).

Microservices example:

An e-commerce platform might split its application into independent microservices — a product catalog service, a cart service, a payment service, and a recommendation engine — each running in its own container. Kubernetes orchestrates these services, scaling the payment service independently during checkout surges while keeping the catalog service at normal capacity, and allowing teams to update the recommendation engine without redeploying the entire application.